

RootLore

2-DAY TECHNICAL INTERVIEW PREP

Evidence-grounded GitHub issue intelligence

A source-verified guide for fresher SDE, backend, full-stack and graduate engineer interviews.



Evidence standard

Current source code is authoritative. README claims were checked against code. Tests, evaluation, and production build were run locally.

Resume caveat

No resume file was present in the supplied attachment directory or repository. This guide therefore does not claim how RootLore is worded on your resume. Compare the recommended pitch with your resume before the interview.

44/44	tests passing	4/4	toy retrieval cases
3	API routes	4	outcomes

1. Project foundation

Simple first

GitHub answers are often scattered across old issues, maintainer comments, docs and releases. RootLore collects a small relevant set, asks an LLM to reason only over it, then checks the model's citations and risky claims before showing an answer. If it cannot support a fix, it improves the report and asks for missing diagnostics.

Interview-ready definition

RootLore is a multi-interface retrieval-augmented issue assistant. A React/Vite web app or Chrome MV3 extension calls an Express API. The API retrieves bounded GitHub evidence, ranks it lexically, sends selected context to Groq's `openai/gpt-oss-20b` under a strict JSON Schema, and deterministically validates the generated result.

Question	Answer
Problem	Project-specific knowledge is fragmented; generic LLMs can hallucinate; issue reports are incomplete.
Users	Developers, issue reporters and maintainers working with a GitHub repository.
Use cases	Find related issues; suggest evidence-backed remediation; improve a report; identify a possible duplicate.
Interesting part	The LLM is not trusted as the final authority: evidence selection and post-generation checks are code-enforced.
Limitations	Lexical—not semantic—retrieval; bounded repo view; process-local caches; no auth/history sync; tiny evaluation set.
Future	Embeddings/hybrid search, shared cache, authentication, broader evaluations, Discussions/PR-diff indexing.

DO NOT SAY THIS

"It scans the whole repository" or "it guarantees the fix." The code reads bounded docs and only explicitly named source files; inferred/tentative actions require review.

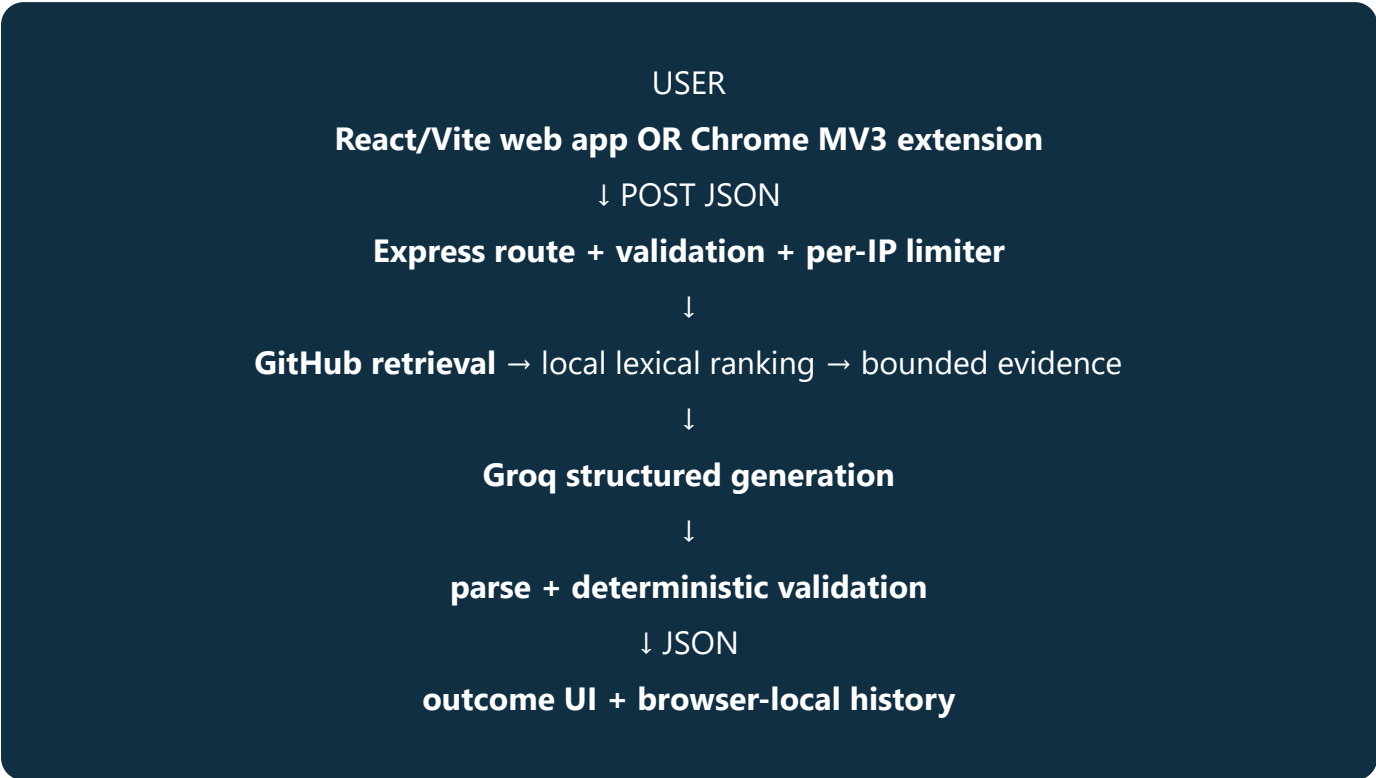
2. Complete tech stack

Technology	What / where	Why	Alternative	Trade-off
JavaScript + ES modules	One language/runtime module system	Backend, web, extension; <code>type: module</code>	TypeScript	Fast iteration; fewer compile-time checks
Node.js 20+	Server JS runtime	Async network-heavy API	Java/.NET	Simple shared language; CPU work can block event loop
Express 5.1	HTTP framework	Routes, JSON, middleware	Fastify/Nest	Small; fewer built-in conventions
React 19 + Vite 8	UI + build tool	SPA form/results/history	Next/Vue	Fast dev/build; client app needs API
Chrome MV3	Extension platform	Reads/inserts GitHub issue content	Userscript	Integrated UX; permissions/DOM fragility
GitHub REST API	Repository HTTP API	Issues/comments/timeline/releases/refs content	GraphQL	Simple endpoints; several calls/rate limits
Groq + gpt-oss-20b	Hosted LLM inference	Structured reasoning over evidence	OpenAI/local model	Speed/JSON schema; provider dependency
JSON Schema	Output contract	Strict response shape	Free text/Zod	Predictable parsing; schema failure possible
cors 2.8	Browser origin policy middleware	Allow configured web origins and extensions	Proxy-only	Simple; extension origin rule is broad
dotenv 16.6	Loads environment secrets	GitHub/Groq keys, port, CORS	Secret manager	Good locally; production needs managed secrets
In-memory Map caches	TTL + dedup	GitHub/Groq response reuse	Redis	No infrastructure; not shared/durable
Node test runner	Built-in tests	44 unit/integration tests	Vitest/Jest	Zero extra runner; simpler ecosystem

Explicitly NOT used

No MongoDB, PostgreSQL, Redis, JWT, server sessions, embeddings, vector database, LangChain or ChromaDB appear in dependencies or source. Browser history uses `localStorage`; extension settings/result use Chrome storage.

3. Architecture



Where logic lives

Layer	Responsibility
src/app.js	CORS, JSON parsing, health route, router, 404 and error middleware.
routes/repositories.js	Request orchestration, validation, optional-evidence degradation and response assembly.
services/github.js	GitHub fetch, normalization, TTL cache, ETag, failures and bounded docs/files.
services/analysis.js	Token/error/numeric matching, ranking, path extraction and command correction.
services/groq.js	Schema/prompt, provider call/cache, parsing, sanitization, outcome decision.
web/src/App.jsx	Repository parsing, submit, results, history and quality display.
extension/*	GitHub DOM extraction/insertion, popup flow, settings and API call.

4. End-to-end analysis flow

1. User enters a GitHub URL, title and description; the web UI parses `owner/repository`. Extension can also send `issueNumber`.
2. `POST /api/repositories/:owner/:repository/analyze`; params come from URL, body from parsed JSON.
3. Limiter counts the client IP; validation checks name regex, strings, non-empty text, title ≤ 200 , description $\leq 10,000$.
4. Fetch recent issues (≤ 100), optional older search (≤ 20), and optional current issue; remove duplicates by issue number.
5. Extract ≤ 3 explicit file paths and rank issues; current issue is forced in, total evidence issues ≤ 3 .
6. `Promise.all` concurrently enriches selected issues plus releases, docs and named files. Optional failures become `[]`.
7. Rank docs/source; combine command corrections + files + docs, deduplicate, slice to ≤ 4 passages.
8. Groq receives bounded evidence and strict schema. Provider JSON is parsed, citations/claims are validated, and an outcome is chosen.
9. API returns evidence, optimization counts, model metadata and analysis; UI renders and stores last 10 analyses locally.

Async interview point

`await` pauses this async function without blocking Node's process. `Promise.all` overlaps independent I/O. Removing `await` would pass unresolved Promises or return too early; one rejection rejects the group, but optional calls are wrapped to resolve as empty arrays.

5. RAG from zero

Three verbs

Retrieve

Find relevant repository evidence.

Augment

Add that evidence to the model prompt.

Generate

Create a structured answer grounded in it.

Grounding ties claims to supplied evidence. **Hallucination** is unsupported generated content. A **context window** is the model's limited input capacity. Retrieval quality determines whether useful evidence reaches it.

Why RootLore qualifies

It retrieves issues, comments, timelines, releases, documentation and explicit files before generation, selects relevant evidence locally, and injects it into the Groq request. It is RAG without vectors.

Actual retrieval type: lexical

Words are lowercased, tokenized (≥ 3 chars), stop words removed, and simple plural endings normalized. Issue score weights title $\times 3$, labels $\times 2$, body $\times 1$ and exact errors $\times 4$; score threshold is 15%. Documents score shared terms plus 3 for numbers within 5%.

Why no embeddings?

For an MVP, lexical scoring was explainable, deterministic, dependency-light and easy to regression-test. The trade-off is missing semantic matches with different vocabulary. Add embeddings later by chunking docs/issues, storing vectors with metadata, retrieving top-k semantic results, then combining normalized lexical and vector scores—while retaining exact-quote validation.

6. GitHub evidence retrieval

Evidence	Actual bound / value	Purpose	Failure
Recent issues	1 page × 100; PRs filtered	Broad current history	Core failure aborts with GitHub status
Older search	≤20; ≤2 signature/title terms	Find matches outside recent window	Optional → empty
Current issue	Exact issue endpoint	Never lose viewed discussion	Optional → null
Comments	≤20 for each of ≤3 issues	Resolution/user evidence	Optional → empty
Timeline	≤100 raw; retain close/reopen/commit/linked PR	Identify closer and links	Optional → empty
Releases	≤10 fetched; ≤3 sent, body ≤400 chars	Connect fixes to versions	Optional → empty
Docs	≤20 files, each ≤100KB, total ≤750KB	Instructions/config/troubleshooting	Optional → empty
Explicit source	≤3 paths; file ≤150KB	Bounded code/data inference	Missing/invalid → empty
Prompt passages	≤4; each ≤600 chars sent	Keep context focused	Continue with remaining evidence

Why bounded?

An entire repository increases GitHub calls, latency, token cost, context noise and prompt-injection surface. Bounded retrieval also makes behavior testable and keeps provider usage predictable.

7. Groq and structured generation

Provider/model: POST <https://api.groq.com/openai/v1/chat/completions>, model [openai/gpt-oss-20b](#), temperature 0.2, max completion 2,200 tokens, timeout 30s.

Input bounds: current issue body 1,000 chars/last 6 comments ×400; other bodies 600/last 3 comments ×300; last 3 timeline events; 3 releases ×400; 4 passages ×600; submitted description sent at ≤5,000 although route accepts ≤10,000.

Schema: summary, improved title/report, questions, duplicate, suggested solution, confidence, issue claims and documentation claims.
strict: `true` reduces parsing ambiguity; local shape checks remain.

Failures: network/timeout →503; non-JSON or invalid shape →502; provider HTTP status preserved. A valid JSON object inside `error.failed_generation` can be recovered with conservative defaults, then fully validated.

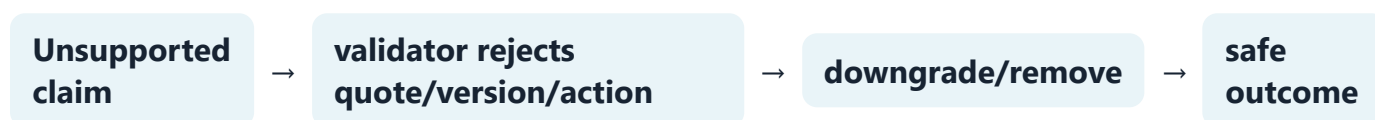
Cache: SHA-256 of report + evidence versions, 24h TTL, ≤250 entries, concurrent identical requests share one Promise.

Could it work without AI?

Partly. Ranking, duplicate candidates, deterministic command correction, quality scoring and issue templates can work without AI. The LLM adds synthesis, cautious inference, follow-up questions and natural rewriting. A non-AI fallback would be less flexible but more predictable.

8. Hallucination controls

- **Evidence-only prompt:** repository text is declared untrusted and cannot override system instructions.
- **Allow-list:** issue numbers and document paths must exist in the supplied evidence.
- **Exact quote:** normalized quote length ≥ 12 and must be a substring of matching evidence.
- **Duplicate:** needs a validated claim marked **duplicate**; current issue cannot match itself.
- **Closure:** closed + not **not_planned** + final closer's comment says fixed/resolved/worked; closure alone is insufficient.
- **Versions:** suggested solution cannot introduce version tokens absent from report/evidence.
- **Risk:** low-confidence destructive/state-changing verbs are blocked without solution evidence.
- **Improved report:** invented environment, reproduction steps and errors are removed; retrieved facts cannot impersonate reporter observations.
- **Context vs decision:** summary-only claims are relabeled as similarity and capped at two; they do not verify cause.



Why prompt rules are insufficient

Models can misunderstand, ignore instructions, or return malformed/crafted content. Deterministic checks are a second, testable boundary. They reduce—not eliminate—hallucination and prompt-injection risk.

Outcome decision matrix

Outcome	Code condition	How to explain
verified_solution	Validated solution claim from issue/docs or deterministic correction/closer confirmation	Repository evidence directly supports the action.
inferred_solution	No direct solution; grounding exists; medium/high confidence; safe solution text	Bounded AI inference, explicitly review-required.
possible_remediation	Safe generated direction remains but is not verified/inferred	Plausible and unverified; low confidence cannot be destructive.
structured_issue	No responsible remediation survives	Collect diagnostics / send improved report.

```
const possibleDuplicate =
  analysis.possibleDuplicate !== currentIssueNumber &&
  allowedNumbers.has(analysis.possibleDuplicate) &&
  evidenceClaims.some(item =>
    item.issueNumber === analysis.possibleDuplicate &&
    item.supports.includes("duplicate"))
  ? analysis.possibleDuplicate : null;
```

Interview warning

A validated exact quote proves the quote exists, not that the repository’s statement is universally correct. The prompt explicitly treats repository text as evidence of intent, not automatic technical truth.

9. API interview table

Method + route	Purpose / input	Validation + limit	Response / failures
GET /health	Liveness; no params/body	No analysis limiter	200 {status: 'ok'}
GET /api/repositories/:owner/:repository/issues	Normalized issues; URL params	Name regex; no custom rate limit	200; 400 invalid; GitHub status or 500
POST /api/repositories/:owner/:repository/issue	Body: title, description, optional integer number	Regex, types, nonempty, 200/10k limits; 10/IP/10min	200 analysis; 400; 429; upstream 4xx/5xx; 502/503

Status reasoning

POST carries a report and triggers a composed analysis; **GET** reads a resource. **400** means client input is invalid. **404** is used by unknown local routes or can pass from GitHub. **429** protects quota. **500** is unexpected server failure; **502** invalid provider output; **503** missing Groq key, timeout or unreachable upstream.

How to secure further

Add authentication/authorization, stricter extension-origin allow-list, request body size at parser level, proxy trust configuration, distributed rate limiting, audit logs, secret manager, outbound allow-list, prompt-injection red teaming and abuse monitoring.

10. Caching and performance

Mechanism	Actual behavior	Limit
Fresh cache	Return Map data within TTL	Process-local
Stale + ETag	Send If-None-Match ; 304 refreshes timestamp	Depends on GitHub ETag
Failure cache	Remember error 60s	Short recovery delay
In-flight dedup	Same URL/analysis shares one Promise	Only one process
GitHub cache	Issues 15m; supporting 6h; docs/source 24h	≤500 entries; oldest insertion removed
Groq cache	Analysis 24h	≤250 entries; oldest insertion removed
Parallel I/O	Nested Promise.all	One unwrapped rejection fails group
Timeouts	GitHub 15s; Groq 30s; extension 45s	Web fetch has no explicit timeout
Rate limit	10 analyses/IP/10m	Map records are not pruned proactively

Multiple backend instances

Each instance has separate cache and limiter state, so hit rate drops, duplicate upstream calls occur, and a client can receive more total allowance. Use Redis (or another shared TTL store) for cached data, locks/request coalescing and atomic distributed counters.

Stale cache

Expired entries are not immediately deleted. A later request revalidates with ETag; if unchanged, GitHub returns 304 and the cached data becomes fresh.

11. Security review

Risk	Current protection	Gap / improvement
Secret exposure	Keys only in backend env; <code>.env</code> ignored	Use cloud secret manager, rotation, least-scope token
CORS	Configured origins; any <code>chrome-extension://</code> allowed	Allow-list exact extension ID; CORS is not authentication
Abuse/DoS	Per-IP analysis limiter, bounds, timeouts, caches	Distributed limiter, parser body limit, quotas, proxy IP config
Input injection	Regex, type, length checks; repo text called untrusted	More prompt-injection tests; isolate/label data
Data leakage	Bounded evidence; keys not sent to client	Private repo content reaches Groq; disclose/consent/redact
Extension permissions	<code>activeTab/storage/scripting</code> ; limited host permissions	Audit DOM injection and production host changes
HTML/script injection	React escapes strings by default	Keep URLs constrained; CSP/headers
Private repos	Token permissions govern access	No per-user auth; server token may over-broaden access

Important nuance

The manifest permits localhost and `https://*.onrender.com/*`. Content script matches GitHub issue URLs. The API's extension CORS branch accepts every extension origin; rate limiting and validation still apply but do not establish identity.

12. Failure and edge cases

Case	Actual result
GitHub core fetch fails / limit	Analyze route returns upstream status; network/timeout is 503; rate-limit message recognized.
Optional search/comments/timeline/releases/docs/file fails	Converted to empty evidence; analysis continues.
Groq absent/unreachable/timeout	503. Invalid structured response is 502; provider status otherwise preserved.
Invalid repo syntax	400 before GitHub. Valid-looking nonexistent/private inaccessible repo passes GitHub error (commonly 404).
Empty/non-string/too long report	400. Title max 200; description max 10,000.
Current issue retrieved as candidate	Forced into evidence but forbidden as its own duplicate.
No/weak evidence	Confidence becomes low; unsafe actions removed; structured issue or cautious remediation.
Huge docs	Only eligible ≤100KB files, ≤20 and ≤750KB; passages bounded.
Missing referenced file	Optional empty; no repository scan fallback.
Duplicate request	GitHub and Groq in-flight/cache reuse when keys match.
Frontend cannot reach API	UI catches and displays fetch error; extension times out at 45s.
CORS rejected	Error middleware returns 403 JSON.

13. Testing and evaluation

44/44

100%

node tests

top-1 on 4 cases only

4/4

PASS

retrieval fixtures

Vite build

Area	What is tested
Analysis (8)	Lexical/error ranking, stop words, document/numeric ranking, paths, source chunks, command typo.
GitHub (2)	Search-term selection.
Groq/validation (21)	Citations, reports, closures, outcomes, versions, unsafe actions, sanitation, parse/recovery.
Quality (6)	Logs, reproduction wording and placeholders.
Rate limit (1)	Allowance rejection.
Route/integration (6)	Mocked pipeline, current issue, 502, cache reuse, 400 and CORS.

Commands

`npm test` runs Node’s test runner. `npm run evaluate` measures top-1 on four labeled synthetic fixtures. `npm run check` chains tests, evaluation and the web production build.

DO NOT SAY THIS

“The model is 100% accurate.” The 4/4 figure tests only the deterministic issue ranker on four easy fixtures. It does not measure generation, real repositories, no-match behavior, citation precision or production latency.

Better evaluation: 5–10 repositories, 10–20 queries each, second-person labels, synonyms/ambiguity/no-match/older cases; report Precision@1, Recall@5, MRR, no-match metrics, groundedness, p50/p95 latency and API/token cost.

14. Debugging stories

All five are hypothetical practice scenarios grounded in current code/tests; git history was not used to claim they happened in production.

Problem	Investigation	Root cause	Fix	Learning
Similar issue ranked too low	Log tokens and score components	Synonyms share no terms	Add hybrid embeddings or domain normalization	Measure retrieval separately
Closed issue shown as a fix	Inspect stateReason, timeline closer and comments	Closure was stale/not_planned or unconfirmed	Require closer-confirmed resolution phrase	State ≠ resolution
Invented error entered improved report	Compare generated fields with submitted report	Model copied retrieved issue data	Sanitize errors/steps/environment against submission	Generated rewriting needs provenance
Burst causes GitHub/Groq quota pressure	Count repeated URLs/cache keys	Concurrent identical calls duplicated	Share in-flight Promise + cache failures briefly	Coalescing matters under load
Valid-looking Groq response breaks UI	Capture raw provider envelope	Malformed/missing structured fields	Strict schema + local parser + controlled 502/recovery	Never trust provider shape

Story delivery

State observable symptom → smallest evidence-gathering step → root cause → exact code-level fix → regression test. Avoid claiming production incidents you cannot prove.

15. Design decisions

Choice	Requirement	Options	Selected	Reason	Trade-off
Node.js	Network-heavy service	Java/.NET/Node	Node	Shared JS + async I/O	CPU work can block
Express	Small REST API	Native/Fastify/Nest	Express	Minimal middleware/routing	Less structure
React/Vite	Interactive SPA	Vanilla/Next/Vue	React/Vite	Component UI + fast build	Separate deployment
REST GitHub	Known resources	GraphQL/clone	REST	Simple/cacheable endpoints	More calls
Groq	Fast hosted generation	Other cloud/local	Groq	Structured model API	Vendor dependency
Lexical	Explainable MVP retrieval	Vector/hybrid	Weighted lexical	Deterministic/testable	Misses semantics
Structured output	Stable UI contract	Free text	JSON Schema	Predictable parsing	Provider schema failures
Post-validation	Untrusted model	Prompt only/human only	Deterministic rules	Testable safety boundary	Rules are incomplete
Bounded evidence	Cost/latency/relevance	Whole repo	Top evidence only	Lower noise/quota	May miss context
Memory cache	Single-instance MVP	Redis/database	Map + TTL	No infrastructure	Not distributed
Promise.all	Independent I/O	Sequential calls	Parallel	Lower latency	Need failure isolation
MV3 extension	In-context GitHub UX	Web only/userscript	Extension	Read/insert issue directly	DOM/permission maintenance
No DB	No server accounts/persistence	SQL/NoSQL	Browser history	Simple/private local history	No sync/analytics

Why not send entire repo?

It adds noise, cost, latency and attack surface. Retrieval is an engineering filter, not just a cost optimization.

16. Code-level walkthrough

```
repositoriesRouter.post(
 ("/:owner/:repository/analyze",
  analysisLimiter,
  async (request, response) => {
    const { owner, repository } = request.params;
    const { title, description, issueNumber } = request.body;
    // validate → retrieve → rank → generate → respond
  }
);
```

- `request.params`: strings captured by `:owner` and `:repository`.
- `request.body`: JSON parsed by `express.json()`.
- `analysisLimiter`: middleware executes before handler and may end with 429.
- `async`: enables `await` and returns a Promise; `try/catch` maps errors to JSON.

```
const issues = [currentIssue, ...recentIssues, ...searchedIssues]
  .filter(Boolean)
  .filter((issue, index, items) =>
    items.findIndex(candidate => candidate.number === issue.number) === index
  );
```

`filter(Boolean)` removes null current issue. The second filter keeps the first item for each issue number. Spread syntax merges arrays. Later `slice(0, 3)/slice(0, 4)` enforce evidence limits.

17–18. JavaScript, Node & CS fundamentals

Concept	RootLore connection
Event loop/promises	Async fetch allows other work while I/O waits; Promise.all overlaps calls.
Modules/collections	ES imports/exports; arrays transform/rank; Map/Set cache and deduplicate.
Errors/HTTP/JSON	try/catch/finally maps failures; fetch serializes external contracts.
DBMS	No server DB; add one for accounts, history, feedback, audit and jobs.
OS	One Node process owns Map state; CPU-heavy work can block its event loop.
Networks	DNS + HTTPS/TCP; REST/statuses; cache/parallelism/timeouts reduce latency.
OOP	Mostly functional modules/plain objects. Closures encapsulate state; no class hierarchy.

What if await is removed?

The variable remains a Promise; array/object consumers may fail or the response may return too early.

19. Limitations and roadmap

Current limitation	Impact	Improvement	Trade-off
Lexical-only	Misses synonym/concept matches	Hybrid BM25/embeddings + reranking	Infra/cost/opacity
Bounded repo understanding	Relevant unreferenced code may be missed	Index repository with freshness metadata	Storage/security/indexing work
Memory cache/limiter	No sharing; reset on restart	Redis + distributed locks/counters	New dependency/operations
Browser-local history	No cross-device sync/accounts	Auth + database	Privacy and security scope
4-case evaluation	Cannot claim real-world accuracy	Diverse labeled benchmark	Labeling cost
No Discussions/full PR diffs	Missed evidence	Add bounded connectors	More calls/context
Server GitHub token	Private access tied to one token	Per-user OAuth/GitHub App scopes	Auth complexity
Broad extension CORS	Any extension origin accepted	Exact extension ID allow-list	Environment-specific config
Web no timeout	Fetch may wait too long	AbortController with UX retry	Tune timeout carefully

Prioritization

First: broaden evaluation so you know the actual failure modes. Second: hybrid retrieval if semantic misses dominate. Third: shared cache/auth only when deployment and user needs justify their operational cost.

20. Question bank — Level 1

15 foundation questions

1. What is RootLore?

Tests: Clarity **Answer:** A GitHub issue assistant grounded in bounded repository evidence with deterministic output checks. **Keywords:** RAG, grounding, validation **Follow-up:** How is it unlike ChatGPT?

2. What problem?

Tests: Product sense **Answer:** Scattered project knowledge, hallucinated generic advice and incomplete reports. **Keywords:** issues, docs, maintainers **Follow-up:** Who benefits?

3. Main user flow?

Tests: System overview **Answer:** Submit report; retrieve/rank; generate; validate; render one of four outcomes. **Keywords:** pipeline **Follow-up:** Where can it fail?

4. Why RAG?

Tests: Concept **Answer:** The model needs current repository-specific context not guaranteed in training. **Keywords:** retrieve/augment/generate **Follow-up:** Why no vectors?

5. Interfaces?

Tests: Scope **Answer:** React web SPA and Chrome MV3 extension call the same Express API. **Keywords:** shared backend **Follow-up:** Why extension?

6. Four outcomes?

Tests: Correctness **Answer:** Verified, inferred, possible remediation, structured issue. **Keywords:** confidence/provenance **Follow-up:** How chosen?

7. Primary evidence?

Tests: Retrieval **Answer:** Issues, comments, timeline, releases, docs and named files. **Keywords:** bounded **Follow-up:** Why not whole repo?

8. Model?

Tests: Recall **Answer:** Groq openai/gpt-oss-20b. **Keywords:** 0.2, schema **Follow-up:** Replaceable?

9. Database?

Tests: Honesty **Answer:** None server-side; Maps cache, localStorage history and Chrome storage. **Keywords:** stateless MVP **Follow-up:** When add DB?

10. Biggest strength?

Tests: Judgment **Answer:** Code-enforced evidence validation after generation. **Keywords:** defense in depth **Follow-up:** Is it foolproof?

11. Biggest weakness?

Tests: Honesty **Answer:** Lexical and bounded retrieval can miss relevant context. **Keywords:** semantic recall **Follow-up:** Improve?

12. What is grounding?

Tests: AI basic **Answer:** Linking output claims to supplied evidence. **Keywords:** citations **Follow-up:** Quote enough?

13. Who uses it?

Tests: Product **Answer:** Reporters and maintainers in GitHub workflows. **Keywords:** developer UX **Follow-up:** Private repos?

14. What is issue quality?

Tests: Feature **Answer:** Six explainable checks scored in the web client. **Keywords:** problem, actual, OS, steps, version, logs **Follow-up:** Can model fake score?

15. Production ready?

Tests: Maturity **Answer:** Strong MVP safeguards/tests, but needs auth, distributed controls and broader eval. **Keywords:** trade-offs **Follow-up:** First priority?

20. Question bank — Level 2

20 implementation questions

1. How are issues ranked?

Tests: Code depth **Answer:** Weighted shared words: title 3, labels 2, body 1, exact error 4. **Keywords:** 15% threshold **Follow-up:** Normalization?

2. How docs rank?

Tests: Retrieval **Answer:** Paragraph chunks; term count +3 for numbers within 5%. **Keywords:** 1200 chars **Follow-up:** Weakness?

3. How older issues?

Tests: API **Answer:** GitHub search with up to two distinctive terms, max 20. **Keywords:** cached **Follow-up:** Rate limit?

4. Why filter PRs?

Tests: GitHub knowledge **Answer:** Issues endpoint also returns pull requests; `pull_request` entries are removed. **Keywords:** normalization **Follow-up:** GraphQL?

5. How paths found?

Tests: Safety **Answer:** Regex extracts ≤ 3 explicit paths from allowed directory/extensions. **Keywords:** no scan **Follow-up:** Missed paths?

6. Docs bounds?

Tests: Numbers **Answer:** 20 files, 100KB each, 750KB total. **Keywords:** 24h cache **Follow-up:** Selection order?

7. What is ETag?

Tests: HTTP **Answer:** Validator reused with If-None-Match; 304 keeps cached data. **Keywords:** revalidation **Follow-up:** TTL?

8. Cache eviction?

Tests: Implementation **Answer:** At capacity delete first inserted Map key. **Keywords:** 500/250 **Follow-up:** LRU?

9. What dedupes requests?

Tests: Concurrency **Answer:** Map stores Promise by URL or analysis hash. **Keywords:** in-flight **Follow-up:** Multi-process?

10. How validate quote?

Tests: Safety **Answer:** Allow-listed source plus normalized exact substring ≥ 12 chars. **Keywords:** deterministic **Follow-up:** Paraphrase?

11. How close confirms fix?

Tests: Logic **Answer:** Final closer actor must author resolution-language comment; not_planned excluded. **Keywords:** timeline **Follow-up:** Reporter vs maintainer?

12. How block self duplicate?

Tests: Logic **Answer:** Candidate number must differ from `currentIssueNumber` and have duplicate claim. **Keywords:** allow-list **Follow-up:** No current number?

13. How sanitize report?

Tests: Provenance **Answer:** Remove unsubmitted versions/environment/actions/errors. **Keywords:** source facts **Follow-up:** Title included?

14. Why schema plus parser?

Tests: Defense **Answer:** Remote constraint and local regression-safe validation. **Keywords:** 502 **Follow-up:** Recovery?

15. Failed generation recovery?

Tests: Resilience **Answer:** Parse JSON object from provider error, add conservative defaults, revalidate. **Keywords:** no second call **Follow-up:** Risk?

16. Rate limiter works how?

Tests: Middleware **Answer:** IP Map record window; increment, headers, reject > 10 . **Keywords:** 429 **Follow-up:** Memory leak/proxy?

17. CORS works how?

Tests: Web **Answer:** No origin, configured origin, or extension scheme allowed; otherwise 403. **Keywords:** browser **Follow-up:** Auth?

18. How UI stores history?

Tests: Frontend **Answer:** Last 10 completed analyses in `localStorage`. **Keywords:** browser-local **Follow-up:** Sensitive data?

19. Extension timeout?

Tests: Reliability **Answer:** `AbortController` at 45 seconds. **Keywords:** service worker **Follow-up:** Web timeout?

20. Optional evidence?

Source verified at commit 7a9cb1c • 04 Sep 2026

Tests: Resilience **Answer:** Catch rejected Promise and return empty array so core pipeline continues. **Keywords:** degradation **Follow-up:** Which call is core?

20. Question bank — Level 3

20 WHY/design questions

1. Why Node.js?

Tests: Trade-off reasoning **Answer:** Shared JS + async I/O; accepted: CPU work can block. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

2. Why Express?

Tests: Trade-off reasoning **Answer:** Minimal middleware/routing; accepted: Less structure. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

3. Why React/Vite?

Tests: Trade-off reasoning **Answer:** Component UI + fast build; accepted: Separate deployment. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

4. Why REST GitHub?

Tests: Trade-off reasoning **Answer:** Simple/cacheable endpoints; accepted: More calls. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

5. Why Groq?

Tests: Trade-off reasoning **Answer:** Structured model API; accepted: Vendor dependency. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

6. Why Lexical?

Tests: Trade-off reasoning **Answer:** Deterministic/testable; accepted: Misses semantics. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

7. Why Structured output?

Tests: Trade-off reasoning **Answer:** Predictable parsing; accepted: Provider schema failures. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

8. Why Post-validation?

Tests: Trade-off reasoning **Answer:** Testable safety boundary; accepted: Rules are incomplete. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

9. Why Bounded evidence?

Tests: Trade-off reasoning **Answer:** Lower noise/quota; accepted: May miss context. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

10. Why Memory cache?

Tests: Trade-off reasoning **Answer:** No infrastructure; accepted: Not distributed. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

11. Why Promise.all?

Tests: Trade-off reasoning **Answer:** Lower latency; accepted: Need failure isolation. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

12. Why MV3 extension?

Tests: Trade-off reasoning **Answer:** Read/insert issue directly; accepted: DOM/permission maintenance. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

13. Why No DB?

Tests: Trade-off reasoning **Answer:** Simple/private local history; accepted: No sync/analytics. **Keywords:** requirement/options/choice **Follow-up:** When would you change it?

14. Why exact quotes?

Tests: Grounding **Answer:** They are mechanically verifiable against retrieved text. **Keywords:** provenance **Follow-up:** Are quotes truth?

15. Why four outcomes?

Tests: UX safety **Answer:** Separate direct support, inference, tentative direction and missing evidence. **Keywords:** calibration **Follow-up:** How map confidence?

16. Why current issue forced in?

Tests: Correctness **Answer:** Its comments may contain resolution even if ranking/search misses it. **Keywords:** extension context **Follow-up:** Self-duplicate?

Source-verified at commit 7a9cb1c • 04 Sep 2026

17. Why cache failures?

Tests: Debugging **Answer:** Cache failures are caused by the test data not being up to date. **Keywords:** cache **Follow-up:** How to debug cache failures?

20. Question bank — Level 4

15 scenario/debugging questions

1. GitHub is slow—what do you do?

Tests: Reliability **Answer:** Use 15s timeout, cache/ETag/dedup; degrade optional sources; observe p95. **Keywords:** timeout, cache **Follow-up:** Retries?

2. Groq is down?

Tests: Dependency **Answer:** Return controlled 503; possible future deterministic fallback/queue. **Keywords:** graceful failure **Follow-up:** Circuit breaker?

3. Two users send same report?

Tests: Concurrency **Answer:** In-flight keys share work in one process; cached result reused. **Keywords:** coalescing **Follow-up:** Across instances?

4. Semantic duplicate missed?

Tests: Retrieval **Answer:** Inspect tokens; add synonym mapping or hybrid embeddings; measure Recall@k. **Keywords:** evaluation **Follow-up:** Reranker?

5. False duplicate returned?

Tests: Safety **Answer:** Require valid duplicate citation; tune ranking/threshold; add negative fixtures. **Keywords:** precision **Follow-up:** Human review?

6. Repo has malicious prompt text?

Tests: Security **Answer:** Treat as untrusted in system prompt and enforce local validators. **Keywords:** prompt injection **Follow-up:** Tool execution?

7. Private repo requested?

Tests: Authorization **Answer:** Server token decides access; current design lacks user-level authorization. **Keywords:** least privilege **Follow-up:** GitHub App?

8. Multiple instances?

Tests: Scaling **Answer:** Move cache/limit/lock state to Redis; add shared observability. **Keywords:** distributed state **Follow-up:** Cache keys?

9. Memory grows?

Tests: Operations **Answer:** Caps exist, but limiter records need cleanup; monitor and prune. **Keywords:** bounded cache **Follow-up:** Eviction policy?

10. Bad JSON body?

Tests: API **Answer:** Express parse error middleware returns 400 JSON. **Keywords:** entity.parse.failed **Follow-up:** Body size?

11. One comment endpoint fails?

Tests: Resilience **Answer:** Optional wrapper yields empty evidence; disclose reduced coverage if added. **Keywords:** partial failure **Follow-up:** Silent degradation?

12. Model invents v9.2?

Tests: Validation **Answer:** Version token absent from evidence causes solution removal. **Keywords:** allow-list **Follow-up:** Non-version numbers?

13. Low-confidence delete advice?

Tests: Safety **Answer:** Unsafe verb pattern blocks it without solution evidence. **Keywords:** destructive **Follow-up:** Regex limitations?

14. Docs exceed bounds?

Tests: Performance **Answer:** Priority selection and byte/file caps; only ranked passages reach prompt. **Keywords:** bounded context **Follow-up:** Relevant late doc?

15. Evaluation says 100%?

Tests: Scientific thinking **Answer:** Report 4/4 only; build diverse held-out set and richer metrics. **Keywords:** sample size **Follow-up:** Confidence interval?

20. Question bank — Level 5

15 skeptical interviewer questions

1. Isn't this just an LLM wrapper?

Tests: Depth **Answer:** No: retrieval, ranking, caching, provenance validation and outcome policy are deterministic application logic. **Keywords:** RAG, guardrails **Follow-up:** Remove LLM?

2. Why trust repository comments?

Tests: Epistemics **Answer:** We do not blindly trust them; source and resolution signals are shown, and inferred claims remain labeled. **Keywords:** provenance **Follow-up:** Malicious content?

3. Exact quote means correct?

Tests: Nuance **Answer:** No; it proves provenance only. Repository text can still be wrong. **Keywords:** source≠truth **Follow-up:** Cross-check?

4. Lexical retrieval is primitive?

Tests: Trade-offs **Answer:** Yes, deliberately simple for MVP explainability; limitation is documented and hybrid search is next. **Keywords:** MVP **Follow-up:** Benchmarks?

5. Four tests prove nothing?

Tests: Honesty **Answer:** They are regression fixtures only; README explicitly says so. **Keywords:** scope **Follow-up:** Better dataset?

6. Your rate limiter is production-grade?

Tests: Honesty **Answer:** No; it is process-local and depends on request.ip/proxy config. **Keywords:** Redis **Follow-up:** Evasion?

7. CORS secures API?

Tests: Security **Answer:** No; CORS is a browser policy, not authentication. **Keywords:** authn/authz **Follow-up:** curl?

8. Why is description 10k but prompt 5k?

Tests: Code reading **Answer:** Route accepts/storage response context, but provider input intentionally truncates; this may omit late details and should be surfaced/improved. **Keywords:** bounds **Follow-up:** Fix?

9. Can validators catch all hallucinations?

Tests: Honesty **Answer:** No; they catch defined classes. Semantic misinterpretation can remain. **Keywords:** defense in depth **Follow-up:** Evals?

10. Why no retry?

Tests: Reliability **Answer:** Retries risk amplifying quota/thundering herd; add bounded backoff only for retryable statuses. **Keywords:** idempotency **Follow-up:** Circuit breaker?

11. Why SHA-256?

Tests: Implementation **Answer:** Stable compact cache identity; not a security claim. **Keywords:** hash **Follow-up:** Collision?

12. Could cache leak private data?

Tests: Security **Answer:** Yes, server-process cache is not user-partitioned; per-user authorization/cache scoping is needed. **Keywords:** tenant isolation **Follow-up:** Encryption?

13. Why client quality score?

Tests: Architecture **Answer:** It is immediate and deterministic, but shared server logic would prevent drift across clients. **Keywords:** duplication **Follow-up:** Extension parity?

14. Why broad extension origins?

Tests: Challenge **Answer:** Convenience for unpacked extension IDs; production should exact-allow-list known ID. **Keywords:** CORS **Follow-up:** Config?

15. What did you personally build?

Tests: Integrity **Answer:** Answer only your actual contribution; distinguish repository facts from personal authorship. **Keywords:** ownership **Follow-up:** Hardest part?

21. Cross-questioning chains I

1. Interviewer: "Just an LLM wrapper." **Candidate:** The differentiator is deterministic retrieval, ranking, caching and post-validation.
Interviewer: "Then why LLM?" **Candidate:** For synthesis and structured rewriting; deterministic pieces still work without it.

2. Interviewer: "Why not embeddings?" **Candidate:** Explainable MVP and measurable behavior.
Interviewer: "So recall is weak?" **Candidate:** Potentially; I would prove the gap on a held-out dataset, then add hybrid retrieval.

3. Interviewer: "Can you guarantee no hallucination?" **Candidate:** No. We reduce defined failure classes.
Interviewer: "What remains?" **Candidate:** Semantic misreading, incorrect repository text and validators' blind spots.

4. Interviewer: "Closed issue means solved?" **Candidate:** No; closure can be stale/duplicate/declined.
Interviewer: "Your rule?" **Candidate:** Exclude not_planned and require the closer's explicit resolution comment.

5. Interviewer: "Why Promise.all?" **Candidate:** Independent I/O overlaps to reduce latency.
Interviewer: "One fails?" **Candidate:** Optional wrappers resolve empty; core unwrapped failures propagate.

6. Interviewer: "Why Redis?" **Candidate:** Not required for one-process MVP.
Interviewer: "When required?" **Candidate:** Multiple instances, durable shared limits, higher cache hit rate and cross-process locks.

7. Interviewer: "CORS is security?" **Candidate:** Only a browser-origin control.
Interviewer: "What secures API?" **Candidate:** Authentication, authorization, quotas and validation.

8. Interviewer: "100% accuracy?" **Candidate:** Only 4/4 toy top-1 retrieval fixtures.
Interviewer: "Credible claim?" **Candidate:** Dataset size + Precision@1/Recall@5/MRR/groundedness and latency.

21. Cross-questioning chains II

9. Interviewer: "Why JSON Schema?" **Candidate:** Stable contract and less ambiguous parsing.

Interviewer: "Enough?" **Candidate:** No; local shape, quote, version and outcome validation remain essential.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

10. Interviewer: "Send the whole repo." **Candidate:** It increases noise, cost, latency and attack surface.

Interviewer: "But you may miss files." **Candidate:** Yes; explicit-path retrieval is a precision/coverage trade-off; indexing can improve recall.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

11. Interviewer: "Private repos?" **Candidate:** Backend token permissions control GitHub access.

Interviewer: "Safe multi-user?" **Candidate:** Not yet; needs per-user OAuth/GitHub App and tenant-scoped caching.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

12. Interviewer: "Model says delete cache." **Candidate:** Low-confidence destructive verbs are blocked without solution evidence.

Interviewer: "Regex foolproof?" **Candidate:** No; expand policy/testing and consider structured action classification.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

13. Interviewer: "GitHub limit exceeded." **Candidate:** Core fetch returns status/message; cached fresh data avoids many calls.

Interviewer: "Improve?" **Candidate:** Backoff, stale-if-error, metrics and authenticated/app tokens.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

14. Interviewer: "Why extension?" **Candidate:** It reads/inserts the issue in context.

Interviewer: "Risk?" **Candidate:** GitHub DOM changes, permissions and exact-origin/API host management.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

15. Interviewer: "No database—backend project?" **Candidate:** Backend value is orchestration, policies, caches and external integrations.

Interviewer: "Add DB for what?" **Candidate:** Accounts, shared history, feedback, audit/evaluation and jobs.

Next push: quantify the trade-off, name the current bound, then propose one measured improvement.

ROOTLORE 15-MINUTE REVISION

Definition: Evidence-grounded GitHub issue assistant that retrieves, ranks, generates and validates.

Problem: scattered repo knowledge + incomplete reports + LLM hallucination.

Web/Extension → Express → GitHub evidence → lexical rank → ≤3 enriched issues + ≤4 passages → Groq schema → deterministic checks → 4 outcomes → UI

Stack: JavaScript ES modules, Node, Express, React/Vite, MV3, GitHub REST, Groq gpt-oss-20b, JSON Schema, cors, dotenv, Node tests.

Top decisions: bounded evidence; explainable lexical scoring; structured output; post-validation; in-memory MVP cache.

Hard questions: not merely wrapper; no guarantee of zero hallucination; no embeddings due MVP trade-off; 4/4 is tiny regression only; multi-instance requires shared state.

Limitations: semantic misses, bounded code view, local caches/limiter, local history/no auth, tiny eval.

Key numbers: issues 100+search 20; selected 3; comments 20; timeline 100; releases 10/3 sent; docs 20/750KB/100KB; source 3/150KB; passages 4; score threshold 15%; number proximity 5%; GitHub/Groq/extension timeouts 15/30/45s; caches 15m/6h/24h; analysis 24h; rate 10/10m; title 200; description 10k (5k sent); output 2200.

60-second pitch

RootLore solves GitHub issues using repository evidence rather than generic model memory. A React app or Chrome extension sends a report to Express. The backend retrieves recent and searched issues plus bounded comments, timelines, releases, docs and explicitly referenced files. It ranks them lexically, sends only the strongest context to Groq using strict structured output, then validates issue numbers, exact quotes, versions, duplicate claims and risky actions. It returns a verified solution, labeled inference, tentative remediation or structured issue. The main trade-off is explainable simplicity versus semantic recall; hybrid retrieval and distributed state are logical next steps.

ROOTLORE RAPID FIRE

Question	1–2 sentence answer
RAG?	Retrieve context, augment prompt, generate answer.
Model?	openai/gpt-oss-20b through Groq.
Temperature?	0.2 for consistency.
Issue ranking?	Title×3, labels×2, body×1, exact error×4.
Threshold?	15% issue relevance.
Recent issues?	Up to 100, one page.
Search results?	Up to 20.
Enriched issues?	At most 3.
Prompt passages?	At most 4.
Named files?	At most 3, explicit only.
Docs cap?	20 files, 750KB total, 100KB each.
Timeouts?	GitHub 15s, Groq 30s, extension 45s.
Rate limit?	10 analyses per IP per 10 minutes.
Cache TTLs?	Issues 15m, support 6h, docs 24h, Groq 24h.
ETag?	Conditional stale-cache revalidation.
In-flight dedup?	Concurrent identical work shares a Promise.
Why POST?	Report body triggers composed analysis.
Why 502?	Invalid provider structure.
Why 503?	Unavailable/misconfigured upstream.
DB?	No server DB; localStorage and Chrome storage only.
Vectors?	Not used; retrieval is lexical.
Duplicate safety?	Allowed issue + duplicate claim + not current issue.
Quote safety?	Exact normalized substring, minimum 12 chars.
Closure safety?	Closer-confirmed resolution; not_planned excluded.
Version safety?	No unseen version in suggested solution.
Unsafe action?	Low-confidence destructive actions removed.
Four outcomes?	Verified, inferred, possible, structured issue.
Tests?	44 passing locally.
Evaluation?	4/4 toy top-1 regression only.
First scale fix?	Shared cache/limiter/locks plus observability.

48-hour study plan & truth checklist

Day 1 — understand

1. Draw the pipeline from memory twice.
2. Read the route, then `analysis.js`, `github.js`, `groq.js`.
3. Explain every bound and outcome aloud.
4. Practice the 15/30/60/120-second pitches.

Day 2 — defend

1. Answer Level 3–5 without notes.
2. Run five cross-question chains aloud.
3. Explain two code snippets line-by-line.
4. Rehearse one hypothetical debugging story and label it honestly.
5. Finish with the 15-minute revision and rapid fire.

Truth checklist

Say

source-verified

, not production-proven. Say

4/4 regression fixtures

, not 100% accurate. Say

reduces hallucinations

, not eliminates them. Say

bounded repository evidence

, not entire repo. Say

functional/modules

, not class-heavy OOP. Never claim personal authorship of a component unless it is true.

Verification performed: `npm test` → 44/44; `npm run evaluate` → 4/4; `npm --prefix web run build` → success. README and code are consistent on these current baselines. Resume representation remains unverified because no resume file was supplied.